
Test Organisation

Who does the testing?

Programmers do the ad hoc testing (and perhaps structured unit level testing)

Programmers, or other team members may do sub-system testing

Independent teams usually do system testing

Users (with support) perform the user acceptance testing

Independent organisations may be called upon to do any of the above testing formally.

Test team roles

Test manager (test lead)

- plan, organise, manage and control the testing
- a project manager for the testing activities

Tester

- prepare test procedures, data, environments, expected results
- execute tests and log incidents
- execute re-tests and regression tests

Test team roles

Test analyst

- to scope the testing, conduct expert interviews
- document test specifications
- liaison with test environment manager, technicians, users, testers
- prepare test reports.

Test automation technician

- record, code and test automated scripts
- prepare test data, test cases and expected results
- execute automated scripts, log results and prepare reports.

Support staff

DBA to help find, extract, manipulate test data

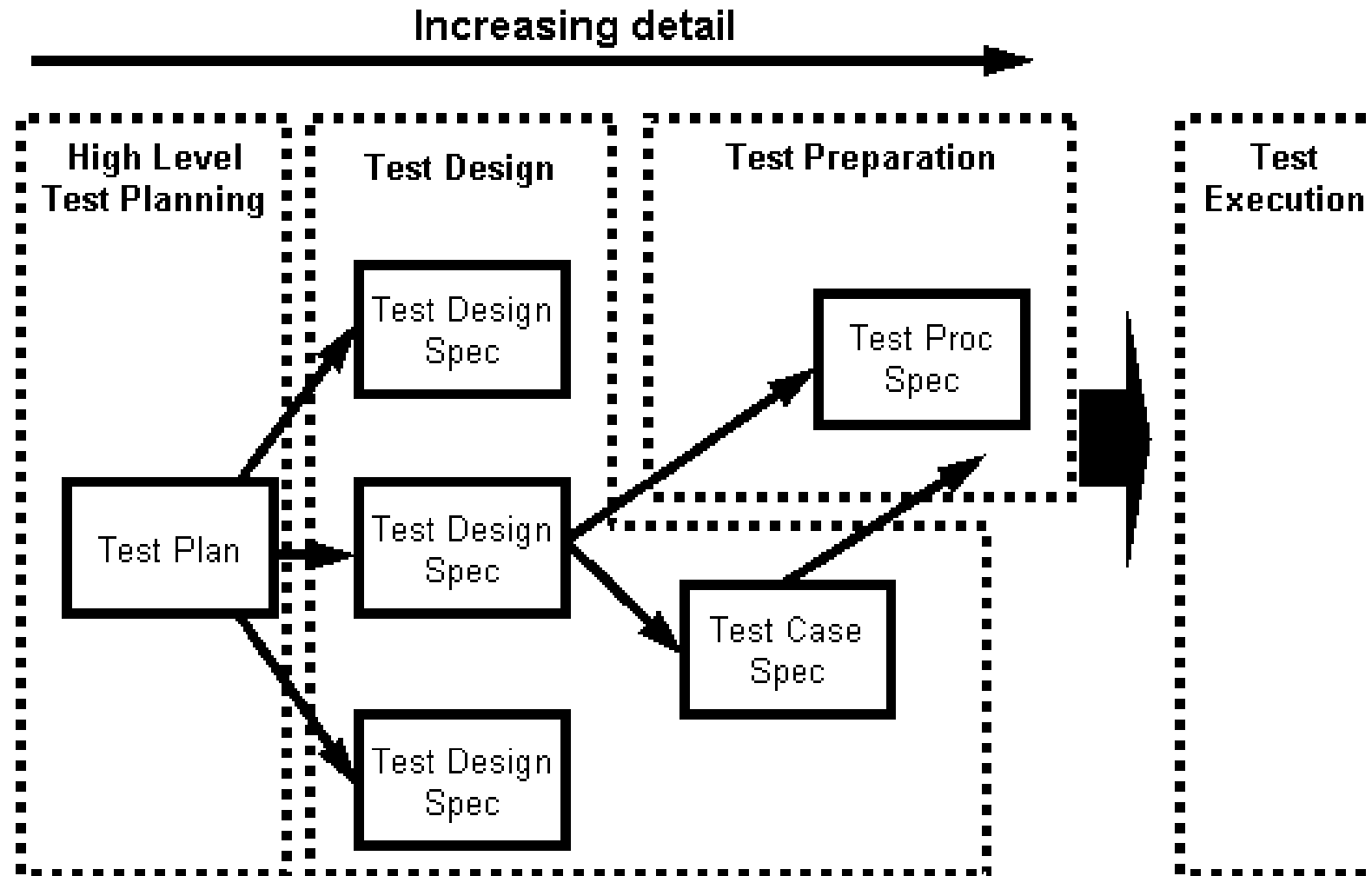
System, network administrators

Toolsmiths to build utilities to extract data, execute tests, compare results etc.

Experts to provide direction
technical
business.

Test Planning and Estimation

Test plans are often layered



Test planning

Planning is influenced by the test policy of the organization, the scope of testing, objectives, risks, constraints, criticality, testability and the availability of resources

Planning and re-planning takes place throughout the project

Test planning is necessarily a non-deterministic exercise where judgement and experience should be complemented with appropriate metrics as a means to managing the uncertainty.

Test planning activities

Defining the overall approach to testing

Integrating testing activities into the overall plan

Scoping, assigning responsibilities, setting entry/exit criteria

Assigning resources for the different tasks defined

Defining the amount, level of detail, structure and templates for the test documentation

Selecting metrics for monitoring and controlling test preparation and execution, defect resolution and risk issues

Setting the level of detail for test procedures in order to provide enough information to support reproducible test preparation and execution.

Exit criteria

The purpose of exit criteria is to define when to stop testing, such as at the end of a test level or when a set of tests has a specific goal.

Typically exit criteria may consist of:

- Thoroughness measures, such as coverage of code, functionality or risk

- Estimates of defect density or reliability measures

- Cost

- Residual risks, such as defects not fixed or lack of test coverage in certain areas

- Schedules such as those based on time to market

Exit Criteria 2

Exit criteria should be objective and measurable

- all tests run successfully

- all faults found are fixed and re-tested

- coverage target (set and) met

- time (or cost) limit exceeded

Coverage items defined in terms of

- requirements, conditions, business transactions

- code statements, branches.

Component Integration test completion criteria

Structural coverage

- 100% coverage of transfers of control or data between components in scope

Functional coverage

- 100% equivalence partitions for all message and function call parameters (valid and invalid values)

- 100% valid transfers of control and data

- 100% invalid/erroneous transfers of control or data

All tests passed without failure

All faults raised, corrected, retested, signed off.

Typical (Functional) System Test completion criteria

Functional coverage

- 100% equivalence partitions

- 100% boundary values

- 100% transaction flows identified in Functional Specification

All tests passed without failure

All incidents raised, corrected, retested, signed off OR waived by Product Assurance Team.

Acceptance Test completion criteria

Acceptance criteria are business oriented

- 100% of all critical system features (Business Requirements Spec and Reqs/Function matrix)

- 100% branch testing through high priority business processes

- 100% process coverage of medium priority business processes

- 100% coverage of low priority business processes (at least once)

All tests (deemed high priority) passed without failure

All incidents raised, corrected, retested, signed off OR waived by Testing Board.

End of Session 1

Any Questions?

Defect Severity & Priority

Defect Severity

Severity: the extent to which the defect can affect the software. Can be of following types (sometimes companies use numbers, i.e. 1-5 severity types):

- (1) Critical: The defect that results in the termination of the complete system or one or more components of the system and causes extensive corruption of the data. There is no acceptable alternative method to achieve the required results.
- (2) Major: The defect that results in the termination of the complete system or one or more components of the system and causes extensive corruption of the data. The failed function is unusable *but there exists an acceptable alternative method* to achieve the required results.
- (3) Moderate: The defect that does not result in the termination, but causes the system to produce incorrect, incomplete or inconsistent.
- (4) Minor: The defect that does not result in the termination and does not damage the usability of the system and the desired results can be easily obtained by working around the defects.
- (5) Cosmetic: The defect that is related to the enhancement of the system where the changes are related to the look and feel of the application.

Defect Priority

Priority: defines the order in which we should resolve a defect. Can be of following types (sometimes companies use numbers, i.e. 1-3 priority types):

- (1) Low: The defect is an irritant which should be repaired, but repair can be deferred until after more serious defect have been fixed.
- (2) Medium: The defect should be resolved in the normal course of development activities. It can wait until a new build or version is created.
- (3) High: The defect must be resolved as soon as possible because the defect is affecting the application or the product severely. The system cannot be used until the repair has been done.

Sometimes, the priority can be escalated upward based on the reporter.

Defect Severity and Priority

High Priority & High Severity: An error which occurs on the basic functionality of the application and **will not allow the user to use the system**.

High Priority & Low Severity: The spelling mistakes that happens on the cover page or heading or title of an application.

High Severity & Low Priority: An error which occurs on the functionality of the application (for which there is no workaround) and will not allow the user to use the system but on click of link which is **rarely used** by the end user.

Low Priority and Low Severity: Any cosmetic or spelling issues which is within a paragraph or in the report (Not on cover page, heading, title).

Decision time

When deadline arrives, may have to stop tests now

- Some faults may be acceptable (for this release)

- Some tests may not be run at all

Might have to relax the exit criteria – assess the risk and agree with customer and document the changes

- (release the software now, but testing could continue)

If exit criteria are met, the software/system can be released to the next test phase or production

What if you finish the test plan very early?

- Have you done enough?

- Strengthen the exit criteria, perhaps and create more tests?

Test Estimation

Why do we need to estimate?

The Project Manager wants you to predict:

- how long testing will take in elapsed time

- how much effort will be needed

- when specialist skills/resources may be required

Resources can be testers, external consultants or services, tools, environments, hardware or software.

What is an estimate?

For each task or activity...

e.g. prepare a test plan, or procedure

What resources are required:

people, tools, test environments etc.

for how long?

what is the resource utilisation? Full time, 50%, occasionally?

what is the lead time for obtaining that resource?

Test Estimation

Estimation is not an exact science

It's an approximate calculation or judgement based on:

- direct experience

- related experience

- informed guesswork.

Usually a 'bottom-up' activity

Break down large complex tasks into smaller, simpler tasks to start with

Estimation 'in the small' is easier to do but....

- do you estimate in days, hours (minutes?)

- it takes more effort to estimate large numbers of tasks

- contingencies add up (you may overdo it)

- some tasks may overlap (double accounting); some tasks may be incomplete (inadequate estimate).

What do we need estimates of?

Just about everything

Need estimates for all explicit activities:

- test planning, design, implementation, execution

- early meetings, test environment, standards/document templates

Also need to consider 'follow-up' activities:

- re-testing, incident reviews, shut-down and clean-up, handover.

What gets forgotten?

Training

'True' cost of tools (selection, pilot, implementation and roll-out)

Relocation of people, desks, comms, pcs etc

Reading time, 'getting up to speed'

Meetings and interviews take elapsed time to organise

'QA' and reviews as zero-duration tasks

Contingency.

Contingency

Contingency is not a cop-out

You never have complete information, so estimation is imprecise and prone to error

Contingency accounts for potential under-estimation

Some tasks may not require contingency

Agree a percentage contingency on the risky tasks e.g. 10% or 15%

Problems in estimating

Total effort for testing is *indeterminate*

- you can't predict how many faults will occur

- you can't predict their severity

- you can't predict time taken to fix

- you can't predict when testing will stop

Allowing enough time to test

Allow for all stages in the test process

Don't underestimate the time taken to set up the testing environment, find data etc.

Testing rarely goes 'smoothly'

- plan for faults to occur

- assume 2-3 passes through system test.

End of Session 2

Any Questions?

Test Approaches (strategies)

Preventative v reactive test approaches

One way to classify test approaches or strategies is based on the point in time at which the bulk of the test design work is begun:

Preventative approaches, where tests are designed as early as possible.

Reactive approaches, where test design comes after the software or system has been produced.

Selecting an approach

The selection of a test approach should consider the context, including:

- The risk of failure of the project, hazards to the product and risks of product failure to humans, the environment and the company

- The skills and experience of the people in the proposed techniques, tools and methods

- The objective of the testing and the mission of the testing team

- Regulatory aspects, such as external and internal regulations for the development process

- The nature of the product and the business.

Test Progress Monitoring and Control

Test progress monitoring

The purpose of test monitoring is to give feedback and visibility about test activities

Information to be monitored may be collected manually or automatically and may be used to measure exit criteria, such as coverage

Metrics may also be used to assess progress against the planned schedule and budget

Common test metrics

Percentage of work done in test case preparation (or percentage of planned test cases prepared)

Percentage of work done in test environment preparation

Test case execution (e.g. number of test cases run/not run, and test cases passed/failed)

Defect information (e.g. defect density, defects found and fixed, failure rate, and retest results)

Test coverage of requirements, risks or code

Testing costs, including the cost compared to the benefit of finding the next defect or to run the next test.

Test Reporting

Test reporting is concerned with summarizing information about the testing endeavour, including:

- What happened during a period of testing, such as dates when exit criteria were met.

- Analyzed information and metrics to support recommendations and decisions about future actions, such as an assessment of defects remaining, the economic

Quantification of the testing

Metrics should be collected during and at the end of a test level in order to assess:

- The adequacy of the test objectives for that test level

- The adequacy of the test approaches taken

- The effectiveness of the testing with respect to its objectives.

Test control

Test control describes any guiding or corrective actions taken as a result of information and metrics gathered and reported.

Actions may cover any test activity and may affect any other software life cycle activity or task.

Examples of test control actions are

- Re-prioritize tests when an identified risk occurs

- Change the test schedule due to availability of a test environment.

- Set an entry criterion requiring fixes to have been retested by a developer before accepting them into a build.

End of Session 3

Any Questions?